

C3 - Intelligence pour la robotique: TP4 – Navigation

Justin Ferdinand

I INTRODUCTION

Ce TP aborde la problématique de la navigation réactive pour un robot mobile évoluant dans un environnement inconnu et complexe. L'objectif est de permettre au robot de se déplacer de manière autonome sans carte préétablie, en utilisant uniquement les informations d'un lidar.

Pour ce faire, nous implémentons l'algorithme "Follow the Gap" (représenté sur la Figure 1) en exploitant les données d'un lidar 2D. Cette méthode consiste à analyser le nuage de points pour détecter les obstacles, définir des bulles de sécurité, et identifier l'espace libre le plus large afin d'y diriger le robot.

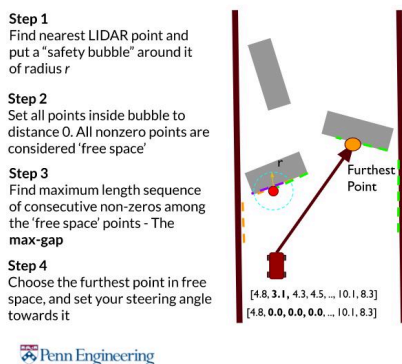


Figure 1: Schéma illustrant l'algorithme Follow The Gap

II MÉTHODOLOGIE

Pour implémenter l'algorithme Follow The Gap, j'ai suivi ce repo Github : (Lien du Git) qui implémente un script python de ce dernier. Les principales étapes de l'algorithme sont les suivantes :

1. **Pré-traitement des données lidar** : Les données brutes du lidar sont filtrées pour réduire le champ de vision à l'avant uniquement et retirer les données aberrantes : valeurs infinies remplacées par une distance maximale plafonnée.

2. **Gestion des obstacles par une bulle de sécurité** : Les bulles de sécurité sont utilisées pour éviter les collisions avec les obstacles détectés par le lidar,
3. **Recherche du plus grand espace libre** : L'algorithme identifie le plus grand espace libre entre les obstacles pour déterminer le vecteur de déplacement optimal du robot,
4. **Commande motrice** : Le point cible est sélectionné au milieu de l'espace libre visé, et les commandes motrices sont appliquées pour diriger le robot vers ce point.

III IMPLÉMENTATION EN PYTHON

III.A PRÉ-TRAITEMENT DES DONNÉES LIDAR

Ces trois fonctions issues du repo git permettent de filtrer les données lidar, de trouver le plus grand gap et de tirer le point cible au milieu de celui-ci.

```
1 def preprocess_lidar(ranges): python
2     """ Preprocess the LiDAR scan
3         array:
4         1. Setting each value to the
5             mean over some window
6         2. Rejecting high values (eg. >
7             3m)
8     """
9     proc_ranges = np.array(ranges)
10    proc_ranges[np.isinf(proc_ranges)]
11    = MAX_LIDAR_DIST
12    proc_ranges[proc_ranges >
13    MAX_LIDAR_DIST] = MAX_LIDAR_DIST
14    return proc_ranges
15
16 def find_max_gap(free_space_ranges):
17     free_space =
18     np.where(free_space_ranges > 0)[0]
19     if len(free_space) == 0:
```

```

14     return None, None
15
16     max_gap = (0, 0)
17     current_gap = (free_space[0],
18                    free_space[0])
19
20     for i in range(1, len(free_space)):
21         if free_space[i] ==
22            free_space[i-1] + 1:
23             current_gap =
24                (current_gap[0],
25                 free_space[i])
26         else:
27             if current_gap[1] -
28                current_gap[0] > max_gap[1]
29                - max_gap[0]:
30                 max_gap = current_gap
31                 current_gap =
32                    (free_space[i],
33                     free_space[i])
34
35     if current_gap[1] - current_gap[0]
36     > max_gap[1] - max_gap[0]:
37         max_gap = current_gap
38
39     return max_gap
40
41 def find_best_point(start_i, end_i,
42                    ranges):
43     """return the index of the point
44     in the middle of the gap"""
45     return int((start_i + end_i) / 2)

```

III.B GESTION DES OBSTACLES : LA BULLE DE SÉCURITÉ

Pour garantir que le robot ne frôle pas les murs du labyrinthe ou les obstacles, j'ajoute une bulle de sécurité autour de chaque obstacle détecté par le lidar. Cette zone de sécurité est créée en invalidant les mesures du lidar qui se trouvent inférieure à une certaine distance du robot. La zone entière devient donc une zone infranchissable pour le robot telle que visible sur la Figure 2.



Figure 2: Affichage des bulles de sécurité autour des obstacles

Le code suivant montre comment j'ai implémenté cette fonctionnalité :

```

1 #trouver le point le plus proche
2 valid_idx =
3 np.where((filtered_ranges > 0) &
4 (~np.isinf(filtered_ranges)))[0]
5 if valid_idx.size > 0:
6     local_min_idx =
7     np.argmin(filtered_ranges[valid_idx])
8     closest_point_idx =
9     int(valid_idx[local_min_idx])
10    min_dist =
11    float(filtered_ranges[closest_point_idx])
12 else:
13     closest_point_idx = None
14     min_dist = None
15
16 #bulle de sécurité sur le point le plus proche
17 if closest_point_idx is not None and
18 min_dist is not None and min_dist > 0:
19
20     #calcul de l'angle de la bulle
21     angle_bubble =
22     math.atan2(BUBBLE_RADIUS,
23                min_dist)
24
25     bubble_radius_idx =
26     int(math.degrees(angle_bubble) /
27         math.degrees(angle_increment))
28
29 #sécurité au cas où
30 if bubble_radius_idx <= 0:
31     bubble_radius_idx =
32     int(np.radians(20) /
33         angle_increment) # ~20 degrés

```

```

19
20     start_bubble = max(0,
21         closest_point_idx -
22         bubble_radius_idx)
23     end_bubble = min(len(proc_ranges)
24         - 1, closest_point_idx +
25         bubble_radius_idx)
26     debug_bubble_indices =
27         range(start_bubble, end_bubble+1)
28     #La région de la bulle vaut 0
29     #pour empêcher le robot d'y aller
30
31     proc_ranges[start_bubble:end_bubble]
32         = 0
33 else:
34     #pas de bulle sans obstacle
35     debug_bubble_indices = []

```

Le code ci-présent permet de trouver le point le plus proche détecté par le lidar, puis de créer une bulle de sécurité autour de ce point en invalidant les mesures du lidar dans cette zone (mise à zéro). La taille de la bulle est déterminée par un rayon défini (BUBBLE_RADIUS) et l'angle correspondant est calculé en fonction de la distance au point le plus proche.

III.C RECHERCHE DU PLUS GRAND ESPACE LIBRE

Une fois les obstacles définis par les bulles de sécurité, j'ai implémenté la recherche du plus grand espace libre entre ces obstacles. Ce gap permet de déterminer la direction dans laquelle le robot doit se diriger. Le code suivant illustre cette étape :

```

1  #on trouve l'écart le plus
2  grand dans les données lidar
3
4  start_gap, end_gap =
5  find_max_gap(proc_ranges)
6

```

```

7  if start_gap is not None and end_gap
8  is not None:
9
10     #le point le plus loin dans
11     l'écart
12
13     best_point_idx =
14     find_best_point(start_gap,
15         end_gap, proc_ranges)
16
17     #ici on va diriger le robot vers
18     ce point
19
20     #calcul de l'angle cible
21
22     angle_target = (fov / 2.0) -
23     (best_point_idx *
24         angle_increment)
25
26
27     #et on fait une commande
28     proportionnelle pour faire une
29     courbe propre
30
31     turn_command = KP_TURN *
32     angle_target
33
34
35     #calcul vitesse des roues
36
37     target_speed_left = VELOCITY_BASE
38     - turn_command
39
40     target_speed_right =
41     VELOCITY_BASE + turn_command
42
43
44     #vitesse max
45
46     max_motor = 20
47
48     target_speed_left = max(-
49         max_motor, min(max_motor,
50             target_speed_left))
51
52     target_speed_right = max(-
53         max_motor, min(max_motor,
54             target_speed_right))

```

Dans le cas où aucun gap n'est trouvé, le robot se dirige vers le côté (gauche ou droite) en fonction de quel côté est le plus libre des deux.

III.D COMMANDE MOTEURS

Dès le départ du TP, je n'ai pas mis en place une commande de moteur faisant une rotation sur place pour se diriger vers la cible. J'ai préféré ajouter un gain KP pour faire une commande pro-

proportionnelle en fonction de la différence angulaire du robot avec sa cible. Cela permet de faire des trajectoires plus douces et sans arrêts. Cela ne permet pas au robot de voir l'ensemble des gaps disponible mais j'avais observé que celui-ci pouvait finir par faire demi-tour si son point de départ était plus libre que son objectif.

IV RÉSULTATS ET ANALYSE

Pour démontrer l'efficacité de l'algorithme Follow The Gap, j'ai réalisé plusieurs tests dans un environnement simulé. Les résultats montrent que le robot est capable de naviguer de manière autonome dans le labyrinthe vide (voir la vidéo MerryGoRound.mp4) en réalisant un tour complet. Je remarque également que celui-ci réalise des trajectoires "optimales" pour suivre le tracé grâce à la commande proportionnelle.

En augmentant la vitesse de rotation et de vitesse linéaire, le robot parvient à naviguer plus rapidement. J'observe toute fois qu'une commande trop brutale peut entraîner une perte dans la navigation, notamment dans les virages serrés. Le robot finit alors par s'arrêter face à un mur sans possibilité d'esquive.

Enfin, j'ai terminé mon analyse de l'algorithme par l'ajout de différents obstacles sur le parcours de Thymio, visibles sur la Figure 3. J'ai ainsi ajouté des animaux et des bouteilles qui peuvent tomber par contact. Dans mon observation, le robot ne touche pas les bouteilles, ce qui confirme l'efficacité de la bulle de sécurité. Il faut ajouter que le robot possède une certaine largeur l'empêchant de passer outre certains obstacles trop proches. Ces observations sont visibles dans la vidéo MerryGoRound_with_objects.mp4.

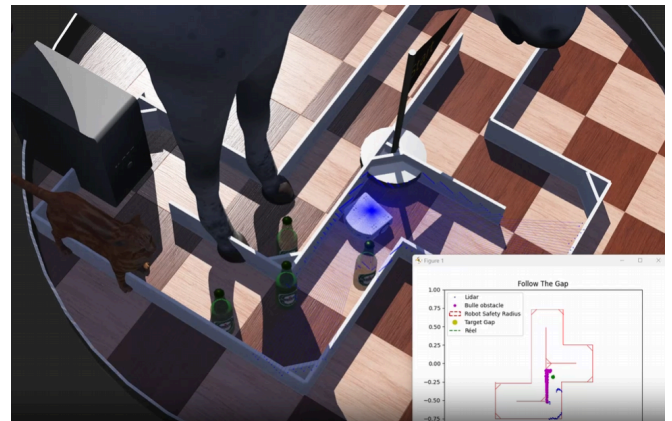


Figure 3: Navigation dans un environnement avec obstacles

V CONCLUSION

Pour conclure, Ce TP m'a permis de comprendre et d'implémenter l'algorithme Follow The Gap pour la navigation réactive d'un robot mobile. En utilisant les données d'un lidar, j'ai pu développer un système capable de détecter les obstacles, de définir des zones de sécurité, et de naviguer efficacement dans un environnement inconnu.

Les résultats que j'observe montrent un algorithme robuste mais qui peut rapidement se perdre lorsque le robot rencontre un obstacle trop proche sans possibilité d'esquiver. J'ajoute ainsi que le retour à l'erreur est difficile dans une situation trop complexe.

Mon implémentation pourrait être améliorée en intégrant un cache des bulles de sécurité pour éviter de recalculer leurs emplacements à chaque itération, évitant ainsi un lag dans la navigation et accélérant la vitesse de calcul. De plus, il serait possible d'intégrer une stratégie de navigation par cycles limites, permettant une trajectoire plus optimale vers la cible par anticipation des obstacles.